

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems **NAME:**K.SARANYA
COURSE CODE: CSA0501 **REG NO:**192424409

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage: 35%

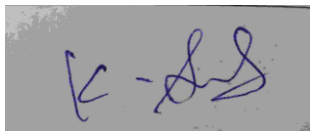
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

*I **K.SARANYA(192424409)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Query Execution Plan & Heuristic Optimization

Consider the following SQL query, which combines a join across two base tables with a correlated aggregate sub-query:

```
SELECT e.name, d.dname
FROM   Employee e, Department d,
      (SELECT dept_id, AVG(salary) AS avgsal
       FROM   Employee
       GROUP BY dept_id) AS S
WHERE  e.dept_id = d.dept_id
      AND e.dept_id = S.dept_id
      AND e.salary > S.avgsal
      AND d.location = 'Chennai';
```

1.1 Step 0 — Initial (Canonical) Query Tree

The parser first produces a canonical tree in which every FROM-list item is combined with a cartesian product and every WHERE condition becomes one large top-level selection.

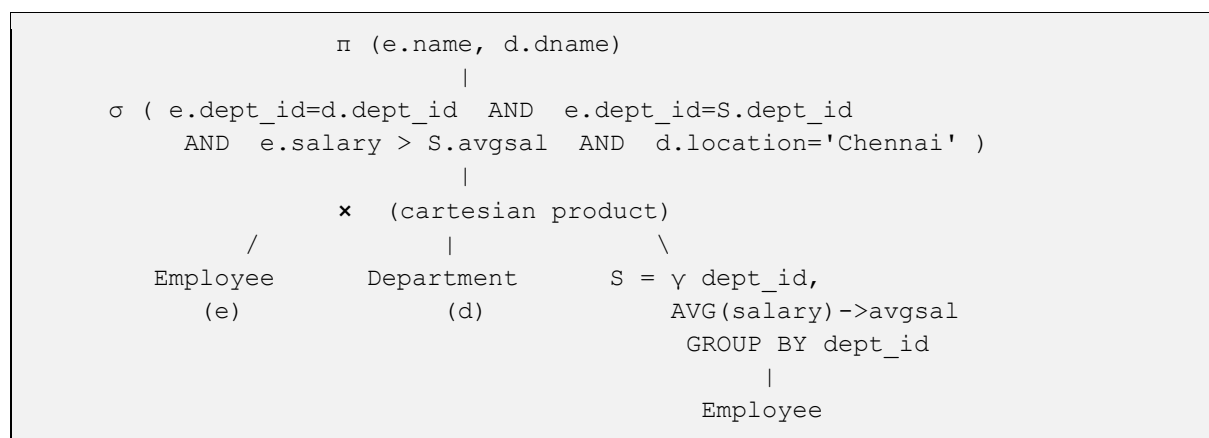


Figure 1.1 — Unoptimized canonical tree: one large cartesian product followed by one large selection.

1.2 Heuristic Rewriting — Step by Step

- Step 1 (Break conjunctive selection): split the single σ with an AND-connected predicate into a cascade of individual σ operators, one per condition, so each can be pushed independently.
- Step 2 (Push selections down): move $\sigma(d.location='Chennai')$ down next to Department, since it only references d; this shrinks Department before any join.
- Step 3 (Push selections down): move $\sigma(e.salary > S.avgsal)$ down to just above the join of Employee and S, since it references only those two operands.
- Step 4 (Replace $\times + \sigma$ with $\bowtie$): wherever a selection's predicate is exactly a join condition ($e.dept_id = d.dept_id$, $e.dept_id = S.dept_id$), combine the cartesian product and the selection into a single join operator $\bowtie$, which the optimizer can implement directly with an efficient join algorithm.
- Step 5 (Push projections down / project early): introduce projections immediately above each leaf, keeping only the attributes needed by later operators (e.g., project Employee down to $\{dept_id, name, salary, dept_id\}$ before joining), so wide unused columns are dropped as early as possible.

- Step 6 (Order joins by selectivity): execute the most restrictive join first — $\sigma(d.location='Chennai')$ typically eliminates most departments, so $Department \bowtie (Employee \bowtie S)$ is evaluated with the filtered, smaller Department operand first, reducing the size of intermediate results feeding the final join.

1.3 Final Optimized Query Tree

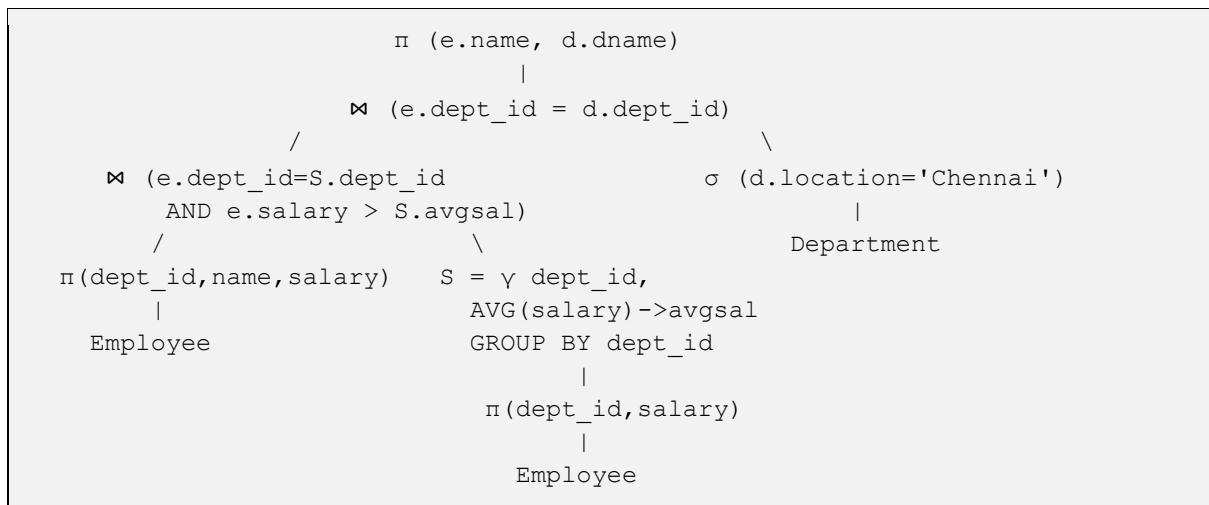


Figure 1.2 — Optimized tree: selections pushed to the leaves, cartesian products replaced by joins, projections applied early, and the most selective join scheduled first.

This rewriting is purely algebraic (heuristic) and is always applied before cost-based physical optimization; it reduces the size of every intermediate relation the cost-based optimizer subsequently has to reason about.

2. Cost-Based Selection: NLJ vs Sort-Merge vs Hash Join

Given relations R and S to be joined, with the statistics below, we compute the estimated I/O cost (in page reads/writes) of three physical join algorithms and select the cheapest.

Parameter	Relation R	Relation S
Tuples	10,000	5,000
Tuples per page	100	100
Pages (P)	100	50

Table 2.1 — Input statistics. Available buffer pages $B = 22$.

2.1 Block Nested Loop Join

To minimize cost, the smaller relation S is chosen as the outer relation, since the number of outer blocks ($B-2$ pages each) multiplies the inner relation's page count.

```
Cost(BNLJ) = P_outer + ceil( P_outer / (B-2) ) * P_inner
            = P(S) + ceil( P(S) / (22-2) ) * P(R)
            = 50 + ceil( 50 / 20 ) * 100
            = 50 + 3 * 100
            = 350 page I/Os
```

2.2 Sort-Merge Join

Both relations are externally sorted (if not already sorted on the join key), then merged in a single pass.

```
-- Sort cost formula: 2 * P * ( 1 + ceil( log(B-1)( ceil(P/B) ) ) )

Sort(R): P=100, B=22
  initial runs   = ceil(100/22) = 5
  merge passes  = ceil( log_21(5) ) = 1
  total passes   = 1 (create runs) + 1 (merge) = 2
  Sort cost(R)  = 2 * 100 * 2 = 400

Sort(S): P=50, B=22
  initial runs   = ceil(50/22) = 3
  merge passes   = ceil( log_21(3) ) = 1
  total passes   = 1 + 1 = 2
  Sort cost(S)   = 2 * 50 * 2 = 200

Merge phase (read each sorted relation once) = P(R) + P(S) = 100 + 50 = 150

Cost(SMJ) = Sort(R) + Sort(S) + Merge = 400 + 200 + 150 = 750 page I/Os
```

2.3 Hash Join (Grace Hash Join)

Applicable since the buffer satisfies $B > \sqrt{\min(P(R), P(S))}$: $22 > \sqrt{50} \approx 7.07$. Each relation is read and written once during partitioning, then read once during the probe phase.

```
Cost(HashJoin) = 3 * ( P(R) + P(S) )  
               = 3 * ( 100 + 50 )  
               = 450 page I/Os
```

2.4 Comparison and Decision

Join Algorithm	Estimated Cost (page I/Os)
Block Nested Loop Join	350
Hash Join	450
Sort-Merge Join	750

Table 2.2 — Cost-based comparison; lowest cost is highlighted by ranking.

The optimizer selects Block Nested Loop Join (350 page I/Os) as the cheapest plan for this input. This outcome occurs because the buffer ($B = 22$ pages) is large relative to the smaller relation S (50 pages) — under a larger workload or a smaller buffer, Hash Join would typically overtake it, since its cost only grows linearly in $P(R)+P(S)$ rather than being bounded by block-count multiplication. In general the optimizer computes all three costs from catalog statistics at compile time and always picks the numerically cheapest option.

3. JDBC Connectivity to MySQL with PreparedStatement

The following Java code establishes a connection to a MySQL database using JDBC, and demonstrates parameterized query execution with PreparedStatement for both a SELECT and an UPDATE, guarding against SQL injection and ensuring resources are closed.

3.1 Java Code

```
import java.sql.*;

public class BankDAO {

    private static final String URL =
"jdbc:mysql://localhost:3306/bankdb?useSSL=false&serverTimezone=UTC";
    private static final String USER = "app_user";
    private static final String PASSWORD = "app_password";

    // Register driver once (optional for JDBC 4.0+, auto-loaded via SPI)
    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBC Driver not found", e);
        }
    }

    // Parameterized SELECT
    public double getBalance(String accountNo) throws SQLException {
        String sql = "SELECT balance FROM accounts WHERE account_no = ?";

        try (Connection conn = DriverManager.getConnection(URL, USER,
PASSWORD);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, accountNo);

            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    return rs.getDouble("balance");
                } else {
                    throw new SQLException("Account not found: " +
accountNo);
                }
            }
        }
    }

    // Parameterized UPDATE inside an explicit transaction
    public void transferFunds(String fromAcc, String toAcc, double amount)
        throws SQLException {
```

```

        String debitSql = "UPDATE accounts SET balance = balance - ? WHERE
account_no = ?";
        String creditSql = "UPDATE accounts SET balance = balance + ? WHERE
account_no = ?";

        try (Connection conn = DriverManager.getConnection(URL, USER,
PASSWORD)) {
            conn.setAutoCommit(false);                                // begin transaction

            try (PreparedStatement debit = conn.prepareStatement(debitSql);
                PreparedStatement credit =
conn.prepareStatement(creditSql)) {

                debit.setDouble(1, amount);
                debit.setString(2, fromAcc);
                int rows1 = debit.executeUpdate();

                credit.setDouble(1, amount);
                credit.setString(2, toAcc);
                int rows2 = credit.executeUpdate();

                if (rows1 == 0 || rows2 == 0) {
                    throw new SQLException("One or both accounts not
found");
                }

                conn.commit();                                // commit both
updates atomically
            } catch (SQLException e) {
                conn.rollback();                                // undo on any
failure
                throw e;
            } finally {
                conn.setAutoCommit(true);
            }
        }
    }
}

```

3.2 Key Points

- PreparedStatement pre-compiles the SQL and binds parameters with setString/setDouble, which prevents SQL injection since parameter values are never concatenated into the query text.
- try-with-resources automatically closes the Connection, PreparedStatement, and ResultSet even if an exception is thrown.
- setAutoCommit(false) plus explicit commit()/rollback() groups the debit and credit updates into a single atomic transaction, consistent with the strict-2PL banking scenario discussed in Question 9.

4. Conflict-Serializability via Precedence Graph

Consider the following interleaved schedule S of four concurrent transactions T1, T2, T3, T4 operating on data items A, B, C:

Step	Operation	Item
1	R1(A)	A
2	R2(A)	A
3	W1(A)	A
4	R3(B)	B
5	W2(B)	B
6	R4(C)	C
7	W3(C)	C
8	W4(A)	A

Table 4.1 — Schedule S, in execution order.

4.1 Identifying Conflicting Operations

Two operations conflict if they belong to different transactions, access the same data item, and at least one is a WRITE. Scanning S in order for each conflicting pair (earlier operation determines the edge direction):

Conflicting Pair	Item	Type	Precedence Edge
R2(A) before W1(A)	A	Read-Write	T2 -> T1
W1(A) before W4(A)	A	Write-Write	T1 -> T4
R2(A) before W4(A)	A	Read-Write	T2 -> T4
W2(B) before R3(B)	B	Write-Read	T2 -> T3
W3(C) before R4(C)	C	Write-Read	T3 -> T4

Table 4.2 — All conflicting operation pairs and the precedence edge each induces.

4.2 Precedence Graph

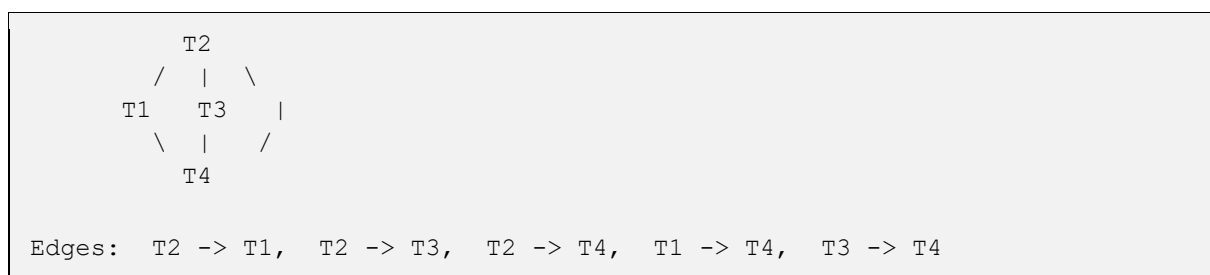


Figure 4.1 — Precedence graph built from Table 4.2.

4.3 Serializability Verdict

The graph has no cycle (every edge points 'forward' from T2 toward T1/T3, and both T1 and T3 point only to T4). By the Precedence Graph Theorem, a schedule is conflict-serializable if and only if its precedence graph is acyclic. Since this graph is acyclic, S is conflict-serializable.

A topological sort of the graph gives an equivalent serial schedule:

Equivalent serial order: T2 -> T1 -> T3 -> T4
--

(T2, T3, T1, T4 is an equally valid topological order, since no edge exists directly between T1 and T3; either serial execution produces the same effect as S.)

5. Basic 2PL: Deadlock Demonstration and Resolution

Consider two transactions operating under Basic Two-Phase Locking:

```
T1:  Lock-X(A);  ... ;  Lock-S(B);  ... ;  Commit
T2:  Lock-X(B);  ... ;  Lock-S(A);  ... ;  Commit
```

5.1 Timeline of Lock Requests

Time	T1	T2	Outcome
t1	Lock-X(A)	-	Granted — T1 holds X(A)
t2	-	Lock-X(B)	Granted — T2 holds X(B)
t3	Lock-S(B) requested	-	BLOCKED — B held X by T2
t4	-	Lock-S(A) requested	BLOCKED — A held X by T1

Table 5.1 — Both transactions are now waiting on each other.

5.2 Deadlock Detection

The resulting Wait-For Graph contains the edges T1 → T2 (T1 waits for the lock T2 holds on B) and T2 → T1 (T2 waits for the lock T1 holds on A):

T1 → T2	T2 → T1 (cycle)
---------	-----------------

Figure 5.1 — A 2-cycle: classic deadlock between two transactions.

5.3 Resolution

```
ON periodic deadlock check:
  cycle <- DetectCycle(WaitForGraph)    // finds {T1 -> T2 -> T1}
  IF cycle EXISTS THEN
    victim <- SelectVictim(cycle)       // e.g., the transaction with
the                                     // smaller amount of work done,
or                                     // simply the younger
transaction
    ABORT(victim)                       // say victim = T2
    ReleaseAllLocks(T2)                 // releases X(B)
    RestartLater(T2)
  END IF

-- T1's pending Lock-S(B) request can now be granted; T1 proceeds and
commits.
-- T2 is restarted from scratch after some delay to avoid immediate re-
collision.
```

This demonstrates the fundamental weakness of Basic 2PL: satisfying the two-phase property alone (all locks acquired before any is released) guarantees conflict-serializability, but it does not prevent deadlocks, so the system still requires an independent deadlock detection-and-recovery mechanism such as periodic Wait-For Graph cycle checking.

6. Wait-Die and Wound-Wait Deadlock Prevention

Both schemes are timestamp-based and non-preemptive-by-symmetry alternatives to detection-based deadlock handling: instead of allowing a deadlock to form and later breaking it, every lock request is resolved immediately using transaction timestamps, so a cycle can never form. Assume:

Transaction	Timestamp TS()	Age
T1	10	Older
T2	20	Younger

6.1 Rules

```
FUNCTION WaitDie(requester Ti, holder Tj)
  IF TS(Ti) < TS(Tj) THEN           // Ti is OLDER than Tj
    Ti WAITS for Tj to release the lock
  ELSE
    Ti DIES (self-aborts) and restarts later with the SAME original
    timestamp
  END IF
END FUNCTION

FUNCTION WoundWait(requester Ti, holder Tj)
  IF TS(Ti) < TS(Tj) THEN           // Ti is OLDER than Tj
    Ti WOUNDS Tj: Tj is forced to abort and release its locks; Ti gets
    the lock
    (unless Tj is already committing, in which case Ti waits briefly)
  ELSE
    Ti WAITS for Tj to release the lock
  END IF
END FUNCTION
```

6.2 Applying Both Schemes to T1 and T2

Scenario	Wait-Die Outcome	Wound-Wait Outcome
T2 (younger) requests item held by T1 (older)	T2 is younger than holder → T2 DIES (aborts, restarts with TS=10 preserved)	T2 is younger than holder → T2 WAITS for T1
T1 (older) requests item held by T2 (younger)	T1 is older than holder → T1 WAITS for T2	T1 is older than holder → T1 WOUNDS T2 (T2 aborted, restarts); T1 proceeds

Table 6.1 — Wait-Die always makes the younger transaction abort; Wound-Wait always makes the younger transaction wait, never the older one.

6.3 Why No Deadlock Can Form

- Wait-Die: only an OLDER transaction is ever allowed to wait, and it waits for a strictly YOUNGER one — timestamps only decrease along any wait chain, so a cycle (which would require an infinite descent) is impossible.

- Wound-Wait: only a YOUNGER transaction is ever allowed to wait, and it waits for a strictly OLDER one — again timestamps strictly decrease along the chain, ruling out cycles.
- Both schemes guarantee freedom from starvation because a restarted transaction keeps its original timestamp, so it eventually becomes the oldest in the system and can no longer be aborted or made to die.

7. Immediate Update Recovery with Checkpointing

The log below (in the order it was written) includes a checkpoint. Under the Immediate Update policy, uncommitted changes may already be on disk, so recovery needs both UNDO (for losers) and REDO (for winners), starting the REDO scan from the checkpoint rather than the beginning of the log.

#	Log Record	Meaning
1	<T1, start>	T1 begins
2	<T1, A, 10, 20>	T1 writes A: old=10, new=20
3	<checkpoint {T1}>	T1 was the only active transaction at checkpoint time
4	<T2, start>	T2 begins
5	<T2, B, 5, 15>	T2 writes B: old=5, new=15
6	<T1, C, 30, 40>	T1 writes C: old=30, new=40
7	<T1, commit>	T1 commits
8	<T3, start>	T3 begins
9	<T3, D, 1, 2>	T3 writes D: old=1, new=2
--	CRASH	System failure — nothing after this point survives

Table 7.1 — Log at the moment of the crash.

7.1 Analysis Pass

Scanning from the checkpoint forward and classifying each transaction by its final status:

Transaction	Status at Crash	Action
T1	COMMITTED (record 7)	REDO all of T1's writes
T2	ACTIVE — no commit/abort record	UNDO all of T2's writes
T3	ACTIVE — no commit/abort record	UNDO all of T3's writes

7.2 Redo Pass (forward, from the checkpoint)

```
REDO T1: A <- 20      (reapply new value from record 2)
REDO T1: C <- 40      (reapply new value from record 6)
-- T2 and T3 are NOT redone: they never committed.
```

7.3 Undo Pass (backward, from the end of the log)

```
UNDO T3: D <- 1      (restore old value from record 9); log <T3, abort>
UNDO T2: B <- 5      (restore old value from record 5); log <T2, abort>
```

```
-- T1 is NOT undone: it already committed and was redone above.
```

7.4 Resulting Database State

Item	Final Recovered Value	Reason
A	20	T1 committed — REDO applied
B	5	T2 uncommitted — UNDO restored old value
C	40	T1 committed — REDO applied
D	1	T3 uncommitted — UNDO restored old value

8. Deferred Update (NO-UNDO/REDO) Recovery

Under Deferred Update, a transaction's writes are held in a private workspace/log buffer and are only applied to the actual database after the transaction commits. Consequently, an uncommitted transaction can never have left any trace on disk, so recovery never needs UNDO — only REDO for committed transactions. Consider the log below:

#	Log Record	Meaning
1	<T1, start>	T1 begins
2	<T1, A, 20>	T1's deferred write: A will become 20
3	<T1, commit>	T1 commits — its writes are now to be applied
4	<T2, start>	T2 begins
5	<T2, B, 15>	T2's deferred write: B will become 15
6	<checkpoint {T2}>	T2 active at checkpoint
7	<T3, start>	T3 begins
8	<T3, C, 5>	T3's deferred write: C will become 5
9	<T2, D, 25>	T2's deferred write: D will become 25
10	<T2, commit>	T2 commits
--	CRASH	T3 never commits

Table 8.1 — Deferred-update log; only the new value needs to be logged (no old value / no UNDO information required).

8.1 Pseudocode

```
ALGORITHM DeferredUpdateRecovery(Log, checkpoint)
BEGIN
    committed <- empty set
    redoList <- empty list of (item, newValue)

    // Pass 1: scan from checkpoint forward, classify transactions
    FOR EACH record FROM checkpoint TO end of log DO
        IF record.type = COMMIT THEN
            committed.add(record.transID)
        END IF
    END FOR

    // Pass 2: scan again, collect writes belonging ONLY to committed
    transactions
    FOR EACH record FROM checkpoint TO end of log DO
```

```

    IF record.type = WRITE AND record.transID IN committed THEN
        redoList.append( (record.item, record.newValue) )
    END IF
END FOR

// Apply REDO only -- no UNDO pass exists in this protocol
FOR EACH (item, newValue) IN redoList DO
    WriteToDisk(item, newValue)
END FOR

-- Any transaction without a COMMIT record (here, T3) contributed
-- NOTHING to disk and requires no rollback action at all.
END

```

8.2 Trace and Result

Transaction	Status	Recovery Action	Result
T1	Committed	REDO A <- 20	A = 20
T2	Committed	REDO B <- 15, REDO D <- 25	B = 15, D = 25
T3	Uncommitted	None (nothing was ever written to disk)	C unaffected

Because no uncommitted transaction's write ever reaches disk under Deferred Update, this protocol is also called NO-UNDO/REDO — it trades a simpler recovery procedure for reduced concurrency, since a transaction's updates cannot be made visible (even to itself, beyond its own buffer) until commit time.

9. Strict 2PL for a Banking System (Fund Transfer vs. Balance Read)

T1 transfers funds from Account A to Account B (debiting A, crediting B); T2 concurrently reads the balance of A. Strict 2PL requires that ALL locks — shared and exclusive — are held until the transaction commits or aborts, which is what prevents T2 from ever observing T1's partially-completed transfer (a dirty read).

9.1 Lock Schedule

Time	T1 (Transfer A→B)	T2 (Read Balance of A)	Outcome
t1	Lock-X(A)	-	Granted
t2	Read A; A <- A - amount	-	In-memory debit (not yet committed)
t3	-	Lock-S(A) requested	BLOCKED — X(A) held by T1 (strict 2PL)
t4	Lock-X(B)	-	Granted
t5	Read B; B <- B + amount	(still waiting)	In-memory credit
t6	COMMIT — release X(A), X(B)	-	Both updates now durable
t7	-	Lock-S(A) granted	T2 proceeds
t8	-	Read A (post-transfer value)	T2 sees the CONSISTENT, post-commit balance

Table 9.1 — T2 is forced to wait until T1's commit, so it can never see the intermediate (post-debit, pre-credit) state.

9.2 Pseudocode

```
PROCEDURE Transfer(T1, fromAcc, toAcc, amount)    // Strict 2PL
  LockManager.acquire(X, fromAcc, T1)            // held until commit
  balanceFrom <- Read(fromAcc)
  IF balanceFrom < amount THEN
    Abort(T1); ReleaseAllLocks(T1); RETURN
  END IF
  Write(fromAcc, balanceFrom - amount)

  LockManager.acquire(X, toAcc, T1)                // held until commit
  balanceTo <- Read(toAcc)
  Write(toAcc, balanceTo + amount)

  Commit(T1)
  ReleaseAllLocks(T1)                            // ONLY now -- this is what makes it
'strict'
END PROCEDURE
```

```

PROCEDURE ReadBalance(T2, acc)                                // Strict 2PL
    LockManager.acquire(S, acc, T2)                            // blocks while T1 holds
X(acc)
    balance <- Read(acc)
    Commit(T2)
    ReleaseAllLocks(T2)
    RETURN balance
END PROCEDURE

```

9.3 Why Strict (Not Just Basic) 2PL Is Used Here

Property	Basic 2PL	Strict 2PL
Serializability	Guaranteed	Guaranteed
Locks may be released before commit?	Yes, once shrinking phase begins	No — all locks held until commit/abort
Dirty reads by other transactions possible?	Possible if a lock is released early and the tx later aborts	Impossible
Cascading aborts?	Possible	Never — the defining benefit for a banking system

For a financial system, avoiding cascading aborts and dirty reads is essential: if T1 released Lock-X(A) right after debiting but before crediting B, T2 could read a balance that reflects money that has 'disappeared', and if T1 subsequently aborted, T2's own decision (already reported to a user) would be based on a value that never truly existed. Strict 2PL closes this window entirely.

10. Pseudocode for Heuristic Query Optimization

This generalizes the step-by-step rewriting demonstrated in Question 1 into a reusable algorithm: given a canonical (unoptimized) query tree, apply selection push-down, early projection, and join reordering by estimated intermediate-result size.

10.1 Pseudocode

```
ALGORITHM HeuristicQueryOptimize(queryTree)
INPUT: canonical relational-algebra tree (cartesian products + one big  $\sigma$  + one  $\pi$ )
OUTPUT: heuristically optimized tree

BEGIN
  // ---- Rule 1: Break conjunctive selections ----
  FOR EACH selection node  $\sigma(p_1 \text{ AND } p_2 \text{ AND } \dots \text{ AND } p_k)$  IN queryTree DO
    REPLACE with a cascade  $\sigma(p_1) \rightarrow \sigma(p_2) \rightarrow \dots \rightarrow \sigma(p_k)$ 
  END FOR

  // ---- Rule 2: Push each selection as far down as possible ----
  FOR EACH selection node  $\sigma(p)$  IN queryTree DO
    target  $\leftarrow$  the node furthest down the tree such that all attributes
      referenced by  $p$  are still available at that point
    MOVE  $\sigma(p)$  to immediately above 'target'
    IF target is a JOIN or CARTESIAN PRODUCT AND
       $p$  references only ONE child's attributes THEN
      PUSH  $\sigma(p)$  further, into that single child subtree
    END IF
  END FOR

  // ---- Rule 3: Combine cartesian product + matching selection into a join ----
  FOR EACH node ( $\sigma(p)$  directly above a  $\times$  node) IN queryTree DO
    IF  $p$  is an equality between attributes of the  $\times$  node's two children
    THEN
      REPLACE ( $\sigma(p)$  over  $\times$ ) WITH  $\bowtie_p$  (a single join operator)
    END IF
  END FOR

  // ---- Rule 4: Push projections down / project early ----
  FOR EACH leaf relation  $L$  IN queryTree DO
    neededAttrs  $\leftarrow$  attributes of  $L$  referenced ANYWHERE above  $L$  in the tree
    INSERT  $\pi(\text{neededAttrs})$  directly above  $L$ 
  END FOR
  FOR EACH internal node  $N$  (join or selection) IN queryTree DO
    neededAttrs  $\leftarrow$  attributes required by  $N$ 's ancestors
    IF neededAttrs is a strict subset of  $N$ 's output attributes THEN
      INSERT  $\pi(\text{neededAttrs})$  directly above  $N$ 
    END IF
  END FOR

  // ---- Rule 5: Order joins by estimated intermediate result size ----
```

```

joinNodes <- all ⋈ nodes IN queryTree
FOR EACH join J IN joinNodes DO
    J.estSize <- EstimateResultSize(J.leftChild, J.rightChild,
J.predicate)
                                // using Statistics: |R|, |S|, and predicate selectivity
END FOR
SORT joinNodes ascending BY estSize
REORDER left-deep join tree so joins with SMALLER estimated results
    are executed FIRST (closest to the leaves), reducing the size
    of every subsequent intermediate relation

RETURN queryTree
END

FUNCTION EstimateResultSize(R, S, predicate)
    IF predicate is an equi-join on key/foreign-key THEN
        RETURN MAX(|R|, |S|)
    ELSE
        RETURN ( |R| * |S| ) / MAX( DistinctValues(joinAttr, R),
                                   DistinctValues(joinAttr, S) )
    END IF
END FUNCTION

```

10.2 Optimization Flow

Canonical Tree	Split Selections	Push Selections Down	$\bowtie + \sigma \rightarrow \bowtie$	Push Projections Down	Reorder Joins by Size	Optimized Tree
----------------	------------------	----------------------	--	-----------------------	-----------------------	----------------

Figure 10.1 — The five heuristic rules applied in sequence, matching the worked example in Question 1.

10.3 Summary of Heuristic Rules

Rule	Goal	Effect on Cost
Split conjunctive selections	Enable independent push-down of each condition	Allows finer-grained placement
Push selections down	Filter tuples as early as possible	Shrinks every relation feeding later operators
Combine $\bowtie + \sigma$ into $\bowtie$	Avoid materializing a full cartesian product	Enables efficient join algorithms (Q2)
Push projections down / project early	Drop unneeded columns immediately	Reduces tuple width and I/O at every stage
Order joins by intermediate size	Keep intermediate relations as small as possible	Minimizes cost of downstream joins

